Pepper's Sanctuary

A rough-but-useful project summary of the Next.js site we built: what the pages do, which functions and methods power them, why those choices were made, and which parts are human-authored versus the kind of machinery nobody sane writes from scratch for fun.

Project type	Main stack	Generated
Next.js App Router site	React, Next.js, CSS, browser APIs	June 6, 2026

Big Picture

The project is a personal hub with several practical pages: a home page, a projects showcase, a Drumroll calendar page, an OSRS stats page, and a CSV merger tool for loot records. It is built with the Next.js App Router, so each page lives in `src/app/.../page.js`, and the shared site styling lives mostly in `src/app/globals.css`.

The site is not just a static portfolio. Some pages are mostly content and layout, but the OSRS and CSV pages have real application behavior: fetching external data, parsing user files, normalizing inconsistent CSV formats, deduplicating loot rows, copying output, and downloading a new CSV file.

Pages We Built

Home Page

`src/app/page.js` acts as the front door. It uses `Link` from Next.js for internal navigation and `usePathname()` to know which route is active. The `PROJECT_PATHS` array keeps the featured project links in one tidy data structure, and `PROJECT_PATHS.map(...)` turns that data into links.

The hero section is custom content: the project path chooser, the 2B-style character asset, the neon visual language, and the personal copy. The repeated navigation pattern is typical app code, but the actual structure and visual intent are hand-authored.

Projects Page

`src/app/projects/page.js` is a showcase page. It has a sticky table of contents powered by `PROJECT_SECTIONS.map(...)`, then two project sections: the Google Form to Discord automation and the Dealersette Discord bot.

This page leans on ordinary React composition: arrays for repeated navigation, JSX for content blocks, external links with `target="_blank"` and `rel="noopener noreferrer"`, and embedded Discord widget markup. The content, links, labels, roles, and project positioning are the human-authored part.

Calendar Page

`src/app/calendar/page.js` embeds a public Google Calendar iframe and adds links to important Google Forms. The important method here is not complicated code; it is integration by embedding an external service. The iframe keeps calendar management inside Google Calendar while still displaying it on the site.

OSRS Stats Page

`src/app/osrs/page.js` is a client-side React page that loads stats for `Bowosette` and `Ironsette`. It uses `useState()` to track loading, error, and loaded data, then `useEffect()` to fetch stats after the page mounts.

The helper `loadOsrsStats(username, type)` calls the local API route instead of calling RuneScape directly from the UI. That keeps the UI simpler and gives us one place to handle OSRS response parsing.

CSV Merger Page

`src/app/csvmerger/page.js` is the most application-like part of the site. It accepts three CSV inputs through textareas or file upload, converts several source formats into a TMB-ready schema, optionally deduplicates rows by player plus item ID, sorts the result by player name, and then lets the user copy or download the merged CSV.

Main Functions and Why They Exist

`parseCsv(text)`

Converts raw CSV text into rows and cells. This is custom because simple `split(",")` breaks on quoted commas, escaped quotes, and line breaks. The function tracks whether it is inside quotes with `inQuotes`, handles doubled quotes, and removes empty trailing rows.

`toCsv(rows)`

Converts arrays back into CSV text. It only wraps values in quotes when needed, and escapes quotes by doubling them. This is the mirror image of parsing: output should be valid CSV, not just strings glued together with commas.

`normalizeHeader(value)`

Lowercases and trims headers so `itemID`, `ItemID`, and stray spaces do not break detection. This is boring but valuable defensive code.

`normalizePlayerName(value)`

Trims a character name and removes the realm suffix after `-`. This turns names like `Name-Realm` into the player name TMB expects.

`getIndex(headers, columnName)`

Finds a column by normalized name. This avoids hard-coding column positions, which matters because RCLootCouncil, Gargul, and fallback data may not arrange columns the same way.

`getValue(row, index)`

Safely reads a cell. If the index does not exist, it returns an empty string. That keeps missing optional columns from exploding the whole merge.

`normalizeDate(value)`

Trims dates and swaps slashes for dashes. This creates a more consistent exported format without trying to fully interpret every possible date style.

`responseToOffspec(value)`

Converts source text into the TMB offspec flag. If a response includes `offspec`, it outputs `1`; otherwise it outputs `0`.

`normalizeRows(rows)`

Detects the source CSV shape and converts it into `dateTime, character, itemID, offspec, id`. This is the heart of the merger because it lets different loot export formats land in one final schema.

`mergeCsvFiles(files, deduplicate)`

Runs the full pipeline: parse each file, normalize rows, remove bad rows, optionally deduplicate, sort A-Z by player name, add the TMB header, and produce both row data and downloadable CSV text.

`updateFile(index, changes)`

Updates one of the three file inputs without mutating React state directly. It uses `setFiles(...map(...))`, which is the normal React-safe way to replace one item inside an array.

`handleUpload(index, event)`

Reads an uploaded CSV with the browser's `File.text()` method, then stores both the file text and the file name in state.

`copyOutput()`

Uses `navigator.clipboard.writeText()` to copy the merged CSV and briefly changes the button label from Copy to Copied. This is small UI feedback with almost no extra complexity.

`downloadOutput()`

Uses `Blob`, `URL.createObjectURL()`, a hidden anchor, and `link.click()` to download the result as `merged.csv`. This is browser machinery: useful, but not the sort of thing a human writes for pleasure every Tuesday.

OSRS API Route

`src/app/api/osrs/route.js` defines a server-side `GET(request)` handler. It reads `username` and `type` from the query string, chooses the normal or ironman hiscore endpoint, fetches RuneScape's plain-text hiscore response, and maps the first skill rows into objects with `name`, `rank`, `level`, and `xp`.

The `SKILLS` array gives meaning to each line of the OSRS response. The RuneScape API returns comma-separated numbers, not friendly objects, so the route turns machine-ish text into data the React page can render directly.

React and Browser Methods Used

- `useState()` stores interactive state: uploaded files, dedupe toggle, copy button label, and OSRS loading/error/data panels.
- `useMemo()` recalculates the CSV result only when inputs or dedupe settings change, instead of recomputing on every render for no reason.
- `useRef()` keeps a reference to the hidden download anchor.
- `useEffect()` loads OSRS stats after the page renders.
- `fetch()` talks to the local API route and, on the server, the OSRS hiscore endpoint.
- `map()`, `filter()`, `flatMap()`, and `sort()` do most of the array transformation work.
- `Set` stores seen loot keys for deduplication.
- `Blob`, `URL.createObjectURL()`, and `navigator.clipboard` provide browser-native copy/download behavior.

Styling and Design Work

`src/app/globals.css` carries the visual identity: dark neon background, sticky nav, glowing project cards, OSRS stat panels, Discord card styling, responsive grids, CSV work panels, and mobile layouts. The CSS uses variables like `--neon-pink`, `--neon-purple`, `--neon-blue`, and `--nav-sticky-clearance` so repeated colors and spacing rules stay consistent.

The responsive behavior is mostly handled with CSS grid, flexbox, `clamp()`, `min()`, and a mobile media query at `640px`. The result is a site that can hold both content pages and tool-like pages without each route needing its own completely separate design system.

Human-Written vs Generated-ish Work

Things you would plausibly write manually

- Project names, descriptions, role labels, and page copy.
- The navigation labels and route choices.
- The list of featured project links and section IDs.
- Which assets to use for the brand, character art, Discord card, and CSV tool card.
- The high-level CSV rules: TMB output headers, dedupe by player plus item, sort by player.
- The Google Calendar iframe URL and Google Form links.

Things you normally would not write manually from memory

- A quote-aware CSV parser that handles commas, line breaks, and escaped double quotes correctly enough for real input.
- The browser download sequence using `Blob`, `URL.createObjectURL()`, a hidden anchor, and cleanup with `URL.revokeObjectURL()`.
- The React state update pattern for replacing one file object without mutating the array.
- The OSRS hiscore parsing pipeline from plain text lines into named skill objects.
- The more intricate CSS layout details: sticky table of contents, responsive project layouts, hover popovers, grid-based panels, and mobile-specific layout changes.
- The repeated security attributes on external links: `target="_blank"` with `rel="noopener noreferrer"`.

Why These Choices Make Sense

- Next.js routes keep each page easy to find: home, projects, calendar, OSRS, CSV merger.
- Client components are used where interaction is needed, especially file upload, copy/download, and OSRS loading state.
- The OSRS API route keeps external parsing out of the UI and gives the front end a clean JSON response.
- The CSV merger uses small helper functions instead of one giant blob of logic, making the pipeline easier to debug.
- CSS variables make the neon identity reusable across routes.
- Browser APIs are used for local file handling so the CSV merger does not need a database or backend upload step.

Rough Current Structure

`src/app/page.js` - home page

`src/app/projects/page.js` - project showcase

`src/app/calendar/page.js` - calendar and forms

`src/app/osrs/page.js` - OSRS UI

`src/app/api/osrs/route.js` - OSRS proxy/parser API

`src/app/csvmerger/page.js` - CSV merger tool

`src/app/globals.css` - shared visual system

`public/` - logos, characters, icons, project images

Short version: the site began as a personal project hub, then grew into a more useful toolkit. The custom parts are the personality, project content, visuals, loot rules, and page intent. The "please let a machine remember this

for me" parts are the browser APIs, CSV edge cases, React state plumbing, and detailed responsive CSS.

Summary generated from the current local source files in `my-site`.